

INSTITUTO TECNOLÓGICO DE VERACRUZ

Nombre de la Asignatura:

- Administración en Base de Datos

Grupo: **AD26 6J3A**

Nombres del alumno:

- Raúl Alberto Guerrero Aguilar
- Num. Control: 23600266

Nombre de la Actividad:

- Reporte de Práctica 6

Carrera: Ingeniería en Sistemas Computacionales

Fecha de Entrega: 07/Octubre/2026

1. Introducción Teórica y Preparación del Entorno

A diferencia de los sistemas de gestión de bases de datos relacionales, en los que la información se organiza en tablas normalizadas cuya integridad referencial se resuelve en tiempo de consulta mediante operaciones de unión (JOIN) y que, por consiguiente, admiten exportaciones lógicas mediante utilidades como `mysqldump` o `pg_dump`, Neo4j implementa un almacenamiento nativo de grafos fundamentado en la **adyacencia libre de índices** (*index-free adjacency*). Bajo este modelo, cada registro de nodo mantiene referencias físicas hacia sus relaciones, y cada relación hacia sus nodos extremos y hacia las relaciones adyacentes, de suerte que el recorrido del grafo se efectúa mediante el seguimiento directo de punteros, con un costo proporcional a la porción visitada y no al volumen total del conjunto de datos. La fidelidad de un respaldo depende, en consecuencia, de que dichos **punteros físicos**, los índices y los **metadatos del esquema** permanezcan mutuamente coherentes, condición que una proyección lógica de los datos no puede garantizar de manera eficiente en escenarios de recuperación ante desastres. Por esta razón, los modelos de grafos nativos requieren **respaldos físicos fuera de línea** (*offline dumps*), que empaquetan los archivos de almacenamiento del motor en un estado consistente y detenido.

Las utilidades administrativas de Neo4j, al igual que cualquier binario en sistemas UNIX/Linux, dependen de la correcta resolución de **variables de entorno**, mecanismo que conviene examinar con rigor. El entorno de un proceso es un bloque de pares nombre-valor que el sistema operativo copia desde el proceso padre hacia cada proceso hijo en el momento de su creación; en virtud de este principio de **herencia unidireccional**, una variable definida en una sesión de terminal solo es visible para los procesos descendientes si ha sido exportada explícitamente mediante `export`, y ninguna modificación efectuada por un proceso hijo se propaga hacia su padre. La **jerarquía de definición** se estructura, por su parte, en capas: archivos de alcance global, como `/etc/environment` y `/etc/profile`; archivos de alcance de usuario, como `~/.profile` y `~/.bashrc`; y asignaciones efectuadas durante la propia sesión, de modo que las capas más específicas y posteriores prevalecen sobre las generales. Dentro de este esquema,

la variable `PATH` define la lista ordenada de directorios en los que el shell busca los ejecutables invocados por nombre, de manera que la persistencia de su configuración en `~/.bashrc` determina que la herramienta `neo4j-admin` resulte invocable de forma reproducible en cada sesión interactiva.

La arquitectura de Neo4j presenta, adicionalmente, una **dependencia crítica sobre la Máquina Virtual de Java (JVM)**. El motor está implementado sobre la plataforma Java, y las utilidades de línea de comandos `neo4j` y `neo4j-admin` constituyen scripts envoltorio (*wrappers*) cuya función consiste en localizar un runtime compatible, mediante la variable `JAVA_HOME`, y lanzar con él el proceso correspondiente. Esta dependencia es **transitiva**: la disponibilidad de cualquier operación administrativa, incluidos el volcado y la restauración, queda supeditada a que el sistema aprovisiona la versión de Java exigida por el motor y a que la variable que la referencia sea resoluble en el contexto de ejecución del shell. El aprovisionamiento deliberado de este entorno constituye, por tanto, una fase fundamental del procedimiento y no un trámite preliminar, pues establece las condiciones deterministas bajo las cuales las fases posteriores de respaldo pueden ejecutarse y repetirse de forma confiable.

Entorno implementado:

- **Oracle VirtualBox y Linux Mint** (Hipervisor y Sistema Operativo invitado).
- **Neo4j Desktop Edición Enterprise** (Aprovisionamiento del DBMS local).
- **OpenJDK 17** (Runtime oficial exigido por el motor).
- **Bash y GNU Nano** (Intérprete de comandos y editor de configuración).

2. Metodología de Implementación

El procedimiento se estructuró en **cinco pasos secuenciales**, cada uno de los cuales satisface una precondition técnica del siguiente: identificación de la instancia objetivo, exposición de los binarios administrativos, aprovisionamiento de la máquina virtual de Java, validación del estado transaccional y, finalmente, ejecución del respaldo con su

auditoría mediante restauración. La Tabla 1 sintetiza la secuencia y su evidencia asociada.

Tabla 1. Resumen del procedimiento y evidencia asociada.

Paso	Objetivo técnico	Comando o artefacto	Evidencia
1	Identificación y aislamiento del DBMS (UUID)	<code>ls</code> sobre el directorio <code>dbmss/</code>	Figura 1
2	Exposición global de binarios	<code>~/ .bashrc</code> y variable <code>PATH</code>	Figura 2
3	Aprovisionamiento de la JVM	<code>openjdk-17-jdk</code> y <code>JAVA_HOME</code>	Figuras 3 y 4
4	Cierre limpio del motor transaccional	<code>neo4j start</code> y <code>neo4j stop</code>	Figura 5
5	Respaldo y auditoría de restauración	<code>neo4j-admin database dump</code> y <code>load</code>	Figura 6

2.1. Paso 1: Identificación y Aislamiento del Entorno DBMS

Neo4j Desktop gestiona múltiples instancias de bases de datos mediante el **aislamiento de cada DBMS** en un directorio propio, identificado con un **identificador único universal (UUID)** y alojado en la jerarquía de configuración del usuario. Esta encapsulación evita la interferencia mutua entre instancias y permite que coexistan versiones, configuraciones y conjuntos de datos diferentes; en contrapartida, impone la determinación inequívoca de la instancia objetivo antes de cualquier operación administrativa, dado que las utilidades de volcado y restauración operan directamente sobre los archivos de la instancia y no sobre una abstracción lógica de la misma.

Comando ejecutado:

```
ls ~/.config/"Neo4j Desktop"/Application/relate-data/dbmss/
```

El comando `ls` lista el contenido del directorio contenedor de los DBMS. El entrecomillado del segmento `"Neo4j Desktop"` impide que el shell fragmente la ruta en dos argumentos a causa del espacio, mientras que la tilde, al permanecer fuera de las comillas, se expande al directorio personal del usuario. El resultado expone el UUID de la instancia, cuyo subdirectorío contiene, entre otros, `bin` (ejecutables), `conf` (configuración), `data`

(almacenamiento) y `logs` (bitácoras), con lo cual quedó delimitada la **ruta exacta del entorno a respaldar** y se estableció el aislamiento necesario para las fases subsiguientes.



```
admindb@admindb-VirtualBox: ~
admindb@admindb-VirtualBox:~$ ls ~/.config/"Neo4j Desktop"/Application/relate-data/dbmss/
dbms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d
dbms-acc8e223-525a-4ad4-9a20-57b54e02f76e
admindb@admindb-VirtualBox:~$ ^C
admindb@admindb-VirtualBox:~$
```

Figura 1. Localización del identificador único (UUID) del DBMS en la jerarquía del sistema de archivos.

2.2. Paso 2: Exposición Global de Binarios (Configuración del PATH)

El archivo `~/.bashrc` es un **script de inicialización** que Bash ejecuta al iniciar cada sesión interactiva y que permite personalizar el entorno del usuario mediante variables, alias y funciones. Su carácter persistente lo convierte en el mecanismo idóneo para extender la variable `PATH`: al anexar el subdirectorio `bin` del motor a la lista de búsqueda, las utilidades `neo4j` y `neo4j-admin` pasan a resolverse por nombre desde cualquier directorio de trabajo, lo que se denomina **ejecución agnóstica de directorio** y elimina la dependencia de rutas absolutas al invocar `neo4j-admin`.

La edición se efectuó con GNU Nano, añadiendo la definición al final del archivo. Dado que un proceso hijo no puede modificar el entorno de su proceso padre, la recarga de la

configuración se realizó mediante `source ~/.bashrc`, instrucción que interpreta el archivo en el contexto de la sesión en curso y hace efectiva la nueva ruta sin necesidad de abrir una terminal adicional. Desde la perspectiva operativa, esta configuración **reduce la variabilidad del procedimiento**: los comandos documentados resultan idénticos independientemente del directorio desde el cual se invoquen, lo que facilita su reproducción y su posterior automatización.

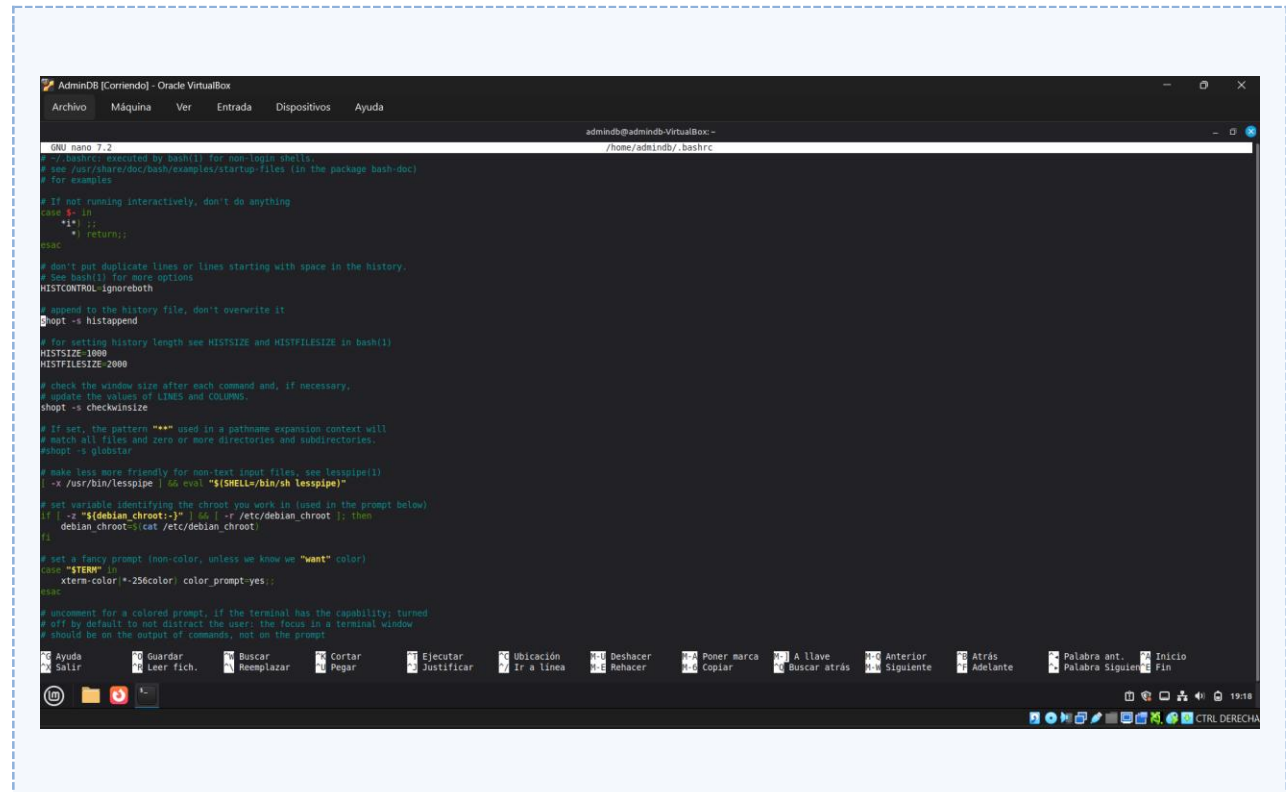


Figura 2. Inyección de la ruta de los binarios de Neo4j en las variables de entorno del shell.

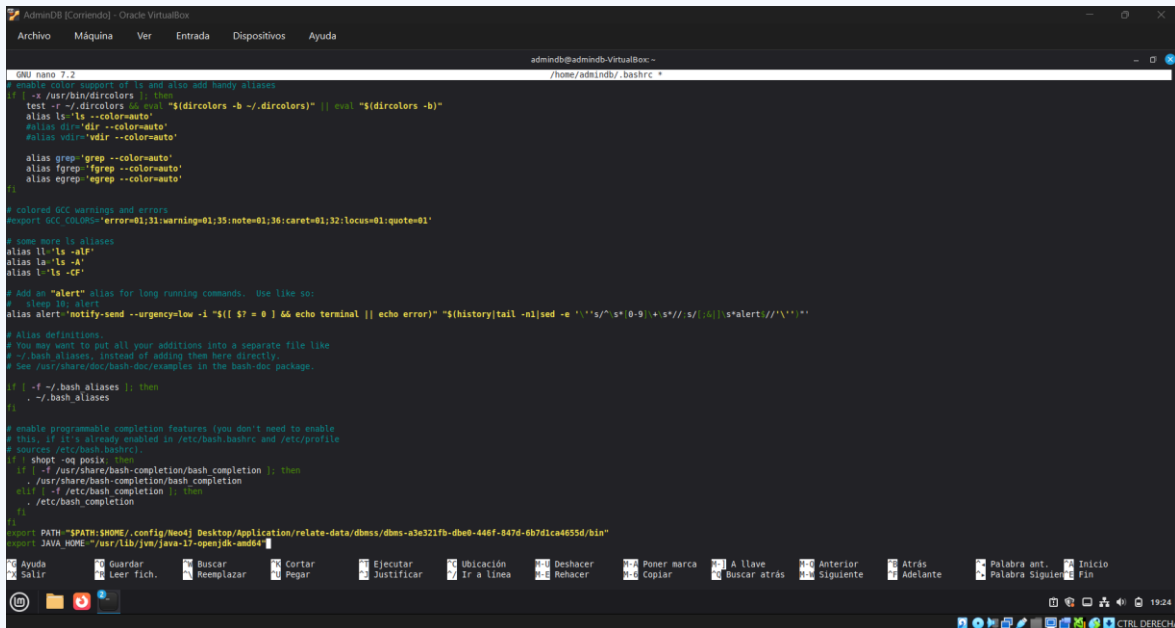
2.3. Paso 3: Aprovisionamiento de la Máquina Virtual de Java (JAVA_HOME)

Las utilidades de línea de comandos de Neo4j son *wrappers* que, antes de ejecutar su lógica, lanzan una JVM resuelta a través de `JAVA_HOME`. Al invocar `neo4j-admin --help` con el entorno aún sin configurar, el motor aplicó su **validación estricta del runtime** y reportó el mensaje `JAVA_HOME is not defined correctly`. Lejos de constituir una anomalía, esta respuesta corresponde a una **salvaguarda de seguridad de diseño**: el motor exige explícitamente **Java 17** y se abstiene de operar sobre un entorno que no puede certificar, lo que impide que una operación administrativa sensible, como un volcado, se ejecute sobre un runtime no soportado.

Validación estricta del runtime

El motor se abstiene de ejecutar operaciones administrativas sin un runtime de **Java 17** resoluble mediante `JAVA_HOME`, lo que protege los procedimientos sensibles frente a entornos no certificados.

Para satisfacer este requerimiento se instaló el paquete `openjdk-17-jdk` mediante el gestor de paquetes APT, que ubica el kit de desarrollo bajo el directorio `/usr/lib/jvm`, y se definió la variable `JAVA_HOME` en `~/.bashrc`, apuntando al directorio raíz de la instalación y exportándola para su herencia por los procesos hijos. Esta combinación opera como una **inyección de dependencias a nivel de sistema operativo**: el binario administrativo no incorpora su propio runtime, sino que lo recibe externamente a través del entorno, de modo que la versión de Java queda desacoplada del ciclo de vida del motor y puede actualizarse de manera controlada sin alterar la instalación de Neo4j. Un efecto de segundo orden es la **portabilidad del procedimiento**: el mismo patrón de inyección mediante variables de entorno se emplea para parametrizar contenedores y servicios en infraestructuras modernas.



```
admin@: ~$ cat ~/.bashrc
# GNU nano 2.2
# enable color support in ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    alias vi='vim --color=auto'
    alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

# some more ls aliases
alias ll='ls -ltp'
alias la='ls -la'
alias l='ls -cf'

# Add an "alert" alias for long running commands. Use like so:
# time ls -l |& alert
alias alert='notify-send --urgency=low -i "${if [ $? = 0 ] && echo terminal || echo error}" "${history}|tail -n1|sed -e "/^s/"s/0-9/+s/// s/;/|/s/alert//"}'"

# Alias definitions
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources (etc/bash.bashrc).
if [ -d /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
else
    . /etc/bash_completion
fi

export PATH=$PATH:$HOME/.config/Neo4j/Desktop/Application/relate-data/dbms/dbms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/bin
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

Figura 3. Configuración permanente de `JAVA_HOME` apuntando a los binarios de OpenJDK 17 en el archivo de entorno.

Tras recargar el entorno de la sesión, la invocación de `neo4j-admin --help` desplegó el menú de ayuda de la utilidad, lo que evidenció la **resolución correcta de ambas variables**, `PATH` y `JAVA_HOME`, y confirmó la habilitación del conjunto de herramientas administrativas.

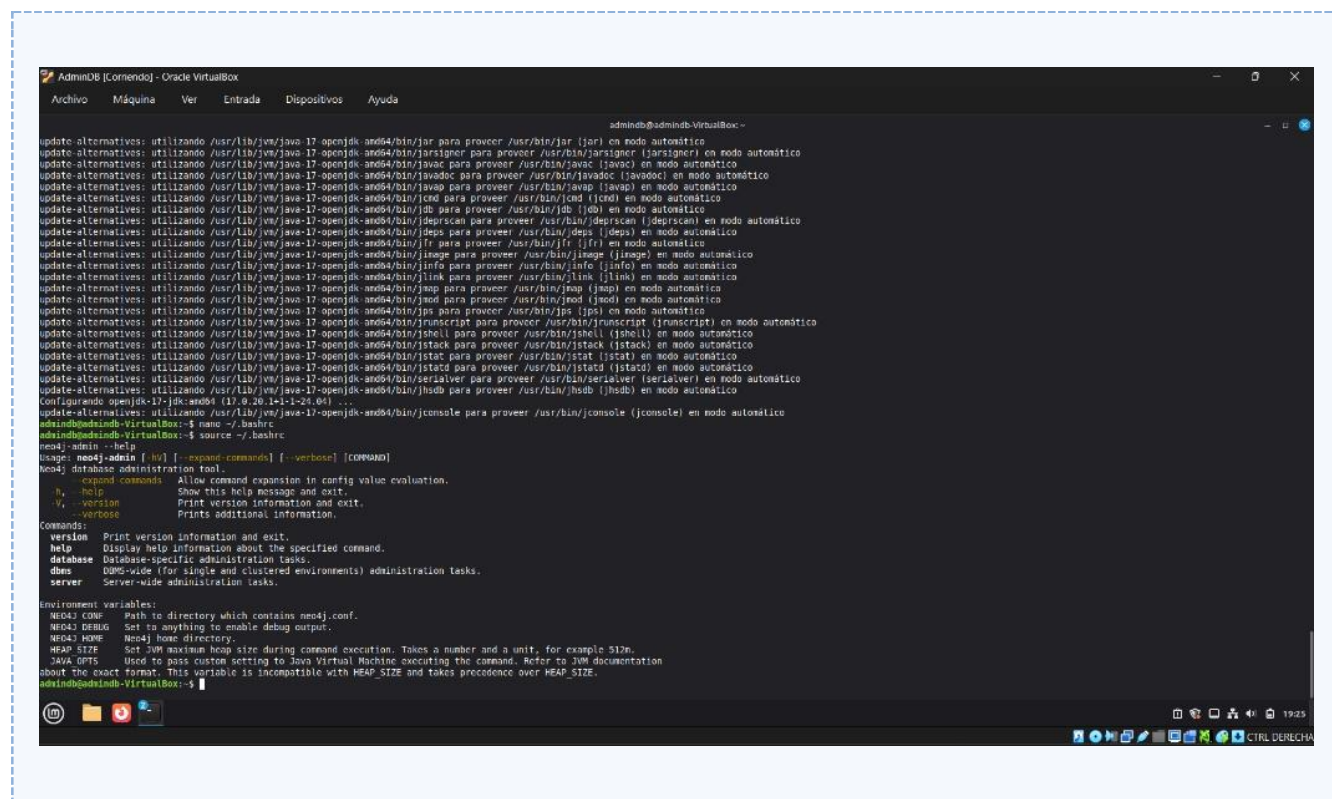


Figura 4. Inicialización exitosa de neo4j-admin tras satisfacer las dependencias arquitectónicas del runtime.

2.4. Paso 4: Auditoría de Estado y Protocolo de Apagado Seguro (Clean Shutdown)

Neo4j garantiza las propiedades ACID mediante **registro anticipado de escritura** (*Write-Ahead Logging*, **WAL**). Conforme a este principio, ninguna modificación se aplica a los archivos de almacenamiento antes de haber sido registrada de forma durable en los **registros lógicos de transacciones** (*transaction logs*): al confirmarse una transacción, sus cambios se inscriben secuencialmente en el log y se sincronizan con el disco, mientras que su aplicación a los archivos de almacenamiento se difiere y se realiza a través de la **caché de páginas** (*page cache*), en la que las páginas modificadas permanecen como páginas sucias hasta ser escritas. Periódicamente, un **punto de control** (*checkpoint*) fuerza la escritura de dichas páginas y registra en el log el punto hasta el cual el almacenamiento es consistente. Esta arquitectura maximiza el rendimiento de escritura, pues sustituye escrituras aleatorias por escrituras secuenciales.

y asegura la durabilidad, dado que el log permite reconstruir cualquier estado posterior al último punto de control.

Principio fundamental del WAL

Ningún cambio alcanza los archivos de almacenamiento sin haber sido registrado antes, de forma durable, en el **log de transacciones**.

De ello se desprende que, entre dos puntos de control, los archivos de almacenamiento pueden no reflejar transacciones ya confirmadas y presentes únicamente en el log y en la memoria. Cuando la utilidad de volcado detecta la condición `Active logical log detected`, es decir, registros lógicos con transacciones pendientes de sincronizar con el disco, **interrumpe la operación por diseño**. Se trata de un **mecanismo de protección del motor**, pues un archivo generado en tales circunstancias capturaría un estado intermedio sin correspondencia con ningún estado transaccional válido y exigiría una recuperación al ser restaurado, con la consiguiente incertidumbre sobre su integridad.

El protocolo de resolución aplicado, estándar en la administración de motores transaccionales, consta de **dos operaciones consecutivas**:

```
neo4j start
neo4j stop
```

La instrucción `neo4j start` inicia el motor, que ejecuta de manera automática su **rutina de recuperación**: reproduce, a partir del último punto de control, las transacciones registradas en los logs lógicos y lleva los archivos de almacenamiento a un estado consistente. La instrucción `neo4j stop` efectúa, a continuación, un **cierre ordenado**: el motor deja de aceptar conexiones, concluye las transacciones en curso, ejecuta un punto de control final, vacía la caché de páginas hacia el disco y libera los bloqueos de archivo, con lo que se alcanza un **estado atómico** sin transacciones pendientes. Los efectos de segundo y tercer orden de este protocolo son relevantes para la estrategia de recuperación: el volcado resultante puede restaurarse sin una fase de recuperación previa, lo que reduce el **tiempo de recuperación (RTO)**; no propaga inconsistencias latentes a la cadena de copias históricas; y deja evidencia verificable del estado

consistente del que partió cada respaldo, requisito habitual en las auditorías de continuidad operativa.

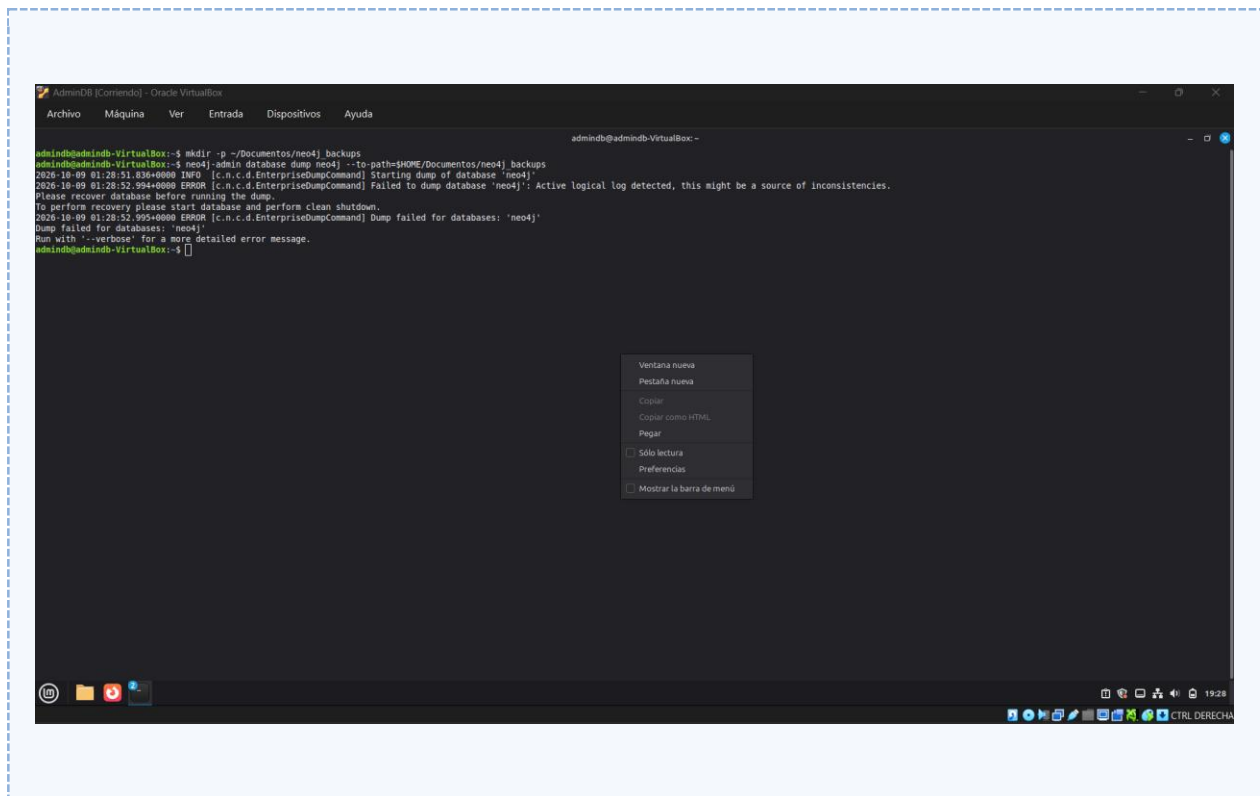


Figura 5. Mecanismo de protección de Neo4j bloqueando un volcado inconsistente y exigiendo un cierre limpio.

2.5. Paso 5: Ejecución Consistente del Respaldo (Dump) y Auditoría de Restauración (Load)

La ejecución definitiva se realizó mediante la utilidad `neo4j-admin` en su **modalidad fuera de línea**, que accede directamente a los archivos de la instancia sin intervención del proceso servidor, lo que elimina la contención con transacciones concurrentes y asegura la atomicidad del archivo generado. Previo a la ejecución se creó el directorio de destino `~/Documentos/neo4j_backups`; a continuación se ejecutaron el volcado de la base de datos y su restauración:

Comandos ejecutados:

```
neo4j-admin database dump neo4j --to-path=$HOME/Documentos/neo4j_backups
neo4j-admin database load neo4j --from-path=$HOME/Documentos/neo4j_backups --
overwrite-destination=true
```

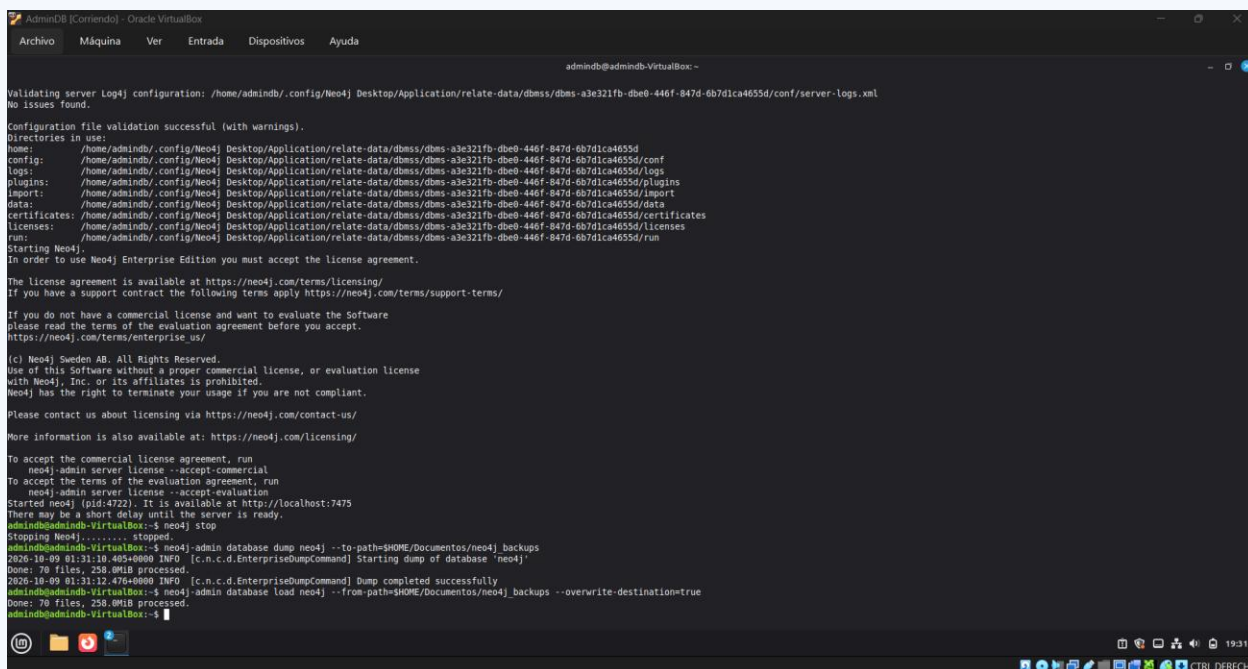
En el primer comando, el argumento `neo4j` designa la base de datos de trabajo y `--to-path` establece el directorio en el que se genera el archivo de volcado, un contenedor único que empaqueta los archivos de la base de datos. En el segundo, `--from-path` indica el directorio de origen y `--overwrite-destination=true` autoriza el reemplazo de los archivos existentes de la base de datos de destino.

La ejecución inmediata de `load` con el parámetro `--overwrite-destination=true` sobre la misma base de datos de la que se obtuvo el volcado constituye una **auditoría de calidad rigurosa** por cuatro razones. En primer lugar, demuestra que el archivo generado es **íntegro y legible**, pues un respaldo solo puede considerarse un control efectivo cuando su restauración ha sido demostrada; sin tal prueba, constituye una hipótesis y no una garantía. En segundo lugar, valida de extremo a extremo la sintaxis, los permisos y el espacio en disco requeridos por el procedimiento. En tercer lugar, la **equivalencia entre el estado original y el restaurado** queda asegurada por construcción, dado que el volcado se obtuvo inmediatamente antes sobre un estado consistente. Por último, el tiempo empleado en procesar **258.0 MiB distribuidos en 70 archivos** proporciona una medición empírica de rendimiento a partir de la cual es posible extrapolar el **tiempo de recuperación (RTO)** para volúmenes de producción, mientras que el **punto de recuperación (RPO)** queda acotado al instante de la última transacción confirmada

antes del apagado limpio. El carácter destructivo de la opción de sobrescritura resulta aceptable únicamente por esta equivalencia previa; en un escenario productivo, dicha restauración se orientaría a una instancia o base de datos de verificación.

Criterio de calidad

Un respaldo solo constituye un control efectivo cuando su **restauración ha sido demostrada**.



```
AdminDB [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

admindb@adminindb-VirtualBox ~
Validating server Log4j configuration: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/conf/server-logs.xml
No issues found.

Configuration file validation successful (with warnings).
Directories in use:
home: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d
config: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/conf
logs: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/logs
plugins: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/plugins
import: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/import
data: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/data
certificates: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/certificates
licenses: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/licenses
run: /home/adminindb/.config/Neo4j/Desktop/Application/relate-data/dbms/dms-a3e321fb-dbe0-446f-847d-6b7d1ca4655d/run

Starting Neo4j.
In order to use Neo4j Enterprise Edition you must accept the license agreement.

The license agreement is available at https://neo4j.com/terms/licensing/
If you have a support contract the following terms apply https://neo4j.com/terms/support-terms/

If you do not have a commercial license and want to evaluate the Software
please read the terms of the evaluation agreement before you accept.
https://neo4j.com/terms/enterprise-us/

(c) Neo4j Sweden AB. All Rights Reserved.
Use of this Software without a proper commercial license, or evaluation license
with Neo4j, Inc. or its affiliates is prohibited.
Neo4j has the right to terminate your usage if you are not compliant.
Please contact us about licensing via https://neo4j.com/contact-us/
More information is also available at: https://neo4j.com/licensing/

To accept the commercial license agreement, run
neo4j-admin server license --accept-commercial
To accept the terms of the evaluation agreement, run
neo4j-admin server license --accept-evaluation
Started neo4j (pid:4722). It is available at http://localhost:7475
There may be a short delay until the server is ready.
adminindb@adminindb-VirtualBox:~$ neo4j stop
Stopping Neo4j..... stopped.
adminindb@adminindb-VirtualBox:~$ neo4j-admin database dump neo4j --to-path=$HOME/Documentos/neo4j_backups
2026-10-09 01:31:18.401+0000 INFO [c.n.c.d.EnterpriseDumpCommand] Starting dump of database 'neo4j'
Done: 70 files, 258.0MiB processed.
2026-10-09 01:31:12.476+0000 INFO [c.n.c.d.EnterpriseDumpCommand] Dump completed successfully
adminindb@adminindb-VirtualBox:~$ neo4j-admin database load neo4j --from-path=$HOME/Documentos/neo4j_backups --overwrite-destination=true
Done: 70 files, 258.0MiB processed.
adminindb@adminindb-VirtualBox:~$
```

Figura 6. Consolidación exitosa: procesamiento atómico de 258 MiB en los ciclos de Dump y Load.

Concluida la verificación, se ejecutó `neo4j start` para reactivar el servicio y reanudar las operaciones normales de la instancia.

3. Conclusiones y Análisis de Resultados

El procedimiento documentado demuestra que el **gobierno de los datos** se materializa en controles técnicos concretos y verificables, y no únicamente en políticas declarativas. La secuencia de validaciones del motor, que comprende la exigencia de un runtime certificado, la verificación del estado de los registros lógicos y el cierre limpio previo al volcado, produce **respaldos de consistencia transaccional verificable**, y la restauración inmediata aporta la evidencia empírica que transforma una copia en un activo de recuperación confiable. Desde esta perspectiva, las protecciones de Neo4j no deben interpretarse como restricciones operativas, sino como **garantías de calidad** que trasladan la detección de inconsistencias del momento crítico de la recuperación al momento controlado del respaldo. Para consolidar este resultado resulta imperativo complementarlo con retención escalonada de copias, resguardo en ubicaciones independientes del sistema productivo, cifrado y control de permisos sobre los archivos de volcado, y alineación con las obligaciones regulatorias de protección de datos personales.

Asimismo, la práctica sustenta una tesis ineludible: el **administrador de bases de datos contemporáneo debe dominar la terminal de Linux**, la manipulación de perfiles de usuario como `~/.bashrc` y la orquestación de binarios sin depender de interfaces gráficas. Las infraestructuras Enterprise se operan de forma remota, mediante sesiones seguras y sin entorno de escritorio, y sus tareas recurrentes se automatizan con programadores como `cron` o temporizadores de `systemd`, lo que exige comprender la herencia de variables de entorno, la resolución del `PATH` y la dependencia de la JVM con la precisión suficiente para diagnosticar y reproducir cualquier procedimiento. Esta competencia es, además, **transferible**: los mismos principios gobiernan la parametrización de contenedores, de pipelines de integración continua y de servicios orquestados, de modo que el dominio de las herramientas nativas habilita la gestión de la infraestructura de datos en su conjunto y no solo la de Neo4j. Finalmente, el requisito de una ventana sin servicio inherente al volcado fuera de línea orienta la evolución natural de la estrategia hacia los **respaldos en línea** y las **topologías en clúster** que ofrece la edición Enterprise, cuando los requerimientos de disponibilidad continua así lo demanden.